ROS 2 Jazzy Environment Validation for an Autonomous Ackermann Vehicle: Hardware and Simulation Platforms

Andrew Abdelmalak
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
andrew.abdelmalak@student.guc.edu.eg

Daniel Boules
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
daniel.boules@student.guc.edu.eg

David Girgis
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
david.girgis@student.guc.edu.eg

Kirolous Kirolous
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
kirolous.kirolous@student.guc.edu.eg

Samir Yacoub
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
samir.yacoub@student.guc.edu.eg

Youssef Salama
Dept. of Mechatronics Engineering
German University in Cairo
Cairo, Egypt
youssef.salama@student.guc.edu.eg

***Abstract*—Autonomous ground vehicles built on Ackermann steering geometry require a robust software middleware layer before any perception, localization, or planning algorithms can be developed. This paper documents the Milestone 1 validation of a ROS 2 Jazzy environment for an autonomous Ackermann vehicle project, targeting two distinct execution contexts: an embedded Raspberry Pi 4 hardware platform and a desktop Gazebo Harmonic simulation platform. We describe the environment provisioning procedure, the custom ROS 2 Python packages developed for validation, and the observable evidence—terminal logs and `ros2 doctor` diagnostics—confirming correct operation. A publisher–subscriber node interacting with a simulated Ackermann vehicle via `/cmd_vel` and `/odom` topics is also presented. All five `ros2 doctor` health checks passed on the hardware platform, and both environments produced expected log output, confirming readiness for subsequent milestones covering localization, navigation, and control.**

***Index Terms*—ROS 2, Jazzy, Ackermann steering, autonomous vehicle, Gazebo Harmonic, Raspberry Pi, embedded robotics, validation**

I. Introduction

Autonomous wheeled vehicles pose a multi-disciplinary engineering challenge spanning perception, localization, planning, and actuation. Modern robotics middleware—most notably the Robot Operating System 2 (ROS 2)—provides the publish-subscribe communication fabric, hardware abstraction, and tooling necessary to integrate these subsystems. Before any algorithm development can begin, however, the middleware itself must be correctly installed, configured, and validated on every target platform.

This paper presents the complete Milestone 1 deliverables for Team 23 in the MCTR1002 Autonomous Systems course at the German University in Cairo (GUC). The goal is to provision and validate a ROS 2 Jazzy environment on two platforms:

1) A *hardware* platform built around a Raspberry Pi 4 single-board computer running Ubuntu Noble 24.04;
2) A *simulation* platform pairing ROS 2 Jazzy with Gazebo Harmonic on a development laptop, providing a physics-simulated Ackermann vehicle model.

Section II reviews relevant literature on Ackermann vehicle autonomy. Section III describes the overall system architecture. Sections IV and V detail the hardware and simulation configurations, respectively. Section VI presents validation results. Section VII discusses limitations and future milestones. Section VIII concludes.

II. Related Work

An autonomous vehicle built around ROS 2 sits at the intersection of four well-studied research modules.

A. System Architecture and Middleware

ROS 2 adopts a decentralized Data Distribution Service (DDS) communications model that decouples high-level tasks (global path planning, SLAM) from low-level controllers (motor drivers, servo actuators), enabling modular development [1].

B. Ackermann Steering Geometry and Kinematics

The classic Ackermann constraint ensures all wheels roll on concentric circular arcs, eliminating tire scrub. The geometric relation is

$$\cot \delta_o - \cot \delta_i = \frac{T}{L}, \tag{1}$$

where δ_o and δ_i are the outer and inner steering angles, T is the track width, and L is the wheelbase [2].

C. Localization

Extended Kalman Filters (EKF) formulated for Ackermann kinematics fuse wheel encoder odometry with IMU data, yielding sub-centimetre positional accuracy when implemented via the `robot_localization` ROS 2 package [3], [4].

D. Navigation and Control

Frenet-frame trajectory planning decouples longitudinal and lateral motion for real-time operation in dynamic environments [5]. Model Predictive Control (MPC) with explicit actuator-delay modelling outperforms classical Pure Pursuit in path-tracking accuracy [6]. Electronic differential control via PID augmented with BP neural networks distributes drive torque across Ackermann-steered axles [7].

III. System Architecture

The Milestone 1 architecture is deliberately minimal: it validates the communication layer before introducing sensor data or planning algorithms. Fig. 1 summarises the component layout.

Hardware Platform (RPi 4)
Ubuntu Noble 24.04 → ROS 2 Jazzy (rmw_fastrtps)
`Validation_Printing_Node` @ 1 Hz

Simulation Platform (ASUS GA401QE)
Ubuntu + ROS 2 Jazzy → Gazebo Harmonic
Ackermann SUV model ↔
`Vehicle_Pub_Sub_Node` (`/cmd_vel` ↑, `/odom` ↓)

Fig. 1. Milestone 1 system architecture.

The hardware and simulation platforms are developed in parallel. Both are self-contained ROS 2 computing nodes; cross-platform DDS discovery is not required for Milestone 1.

IV. Hardware Environment Setup and Validation

A. Platform Specification

The hardware platform is a Raspberry Pi 4 (4 GB RAM) running Ubuntu Noble 24.04 (ARM64). ROS 2 Jazzy was installed via the official APT repository (`ros-jazzy-desktop`) and sourced via `/opt/ros/jazzy/setup.bash`.

B. Environment Diagnostics

The built-in `ros2 doctor` tool evaluates the ROS 2 installation across five health checks: network interface, RMW implementation, ROS 2 packages, Python version, and middleware configuration. Table I reports the diagnostic results.

Running `ros2 doctor --report` confirmed: distribution = `jazzy`, release = `6.8.0-1048-raspi`, middleware = `rmw_fastrtps_cpp`. Fig. 2 shows the terminal output.

TABLE I
ROS2 DOCTOR DIAGNOSTIC RESULTS (HARDWARE PLATFORM)

Check	Status
Network interface	PASS
RMW implementation	PASS
ROS 2 package versions	PASS
Python environment	PASS
Middleware configuration	PASS
Overall	**5/5 PASS**

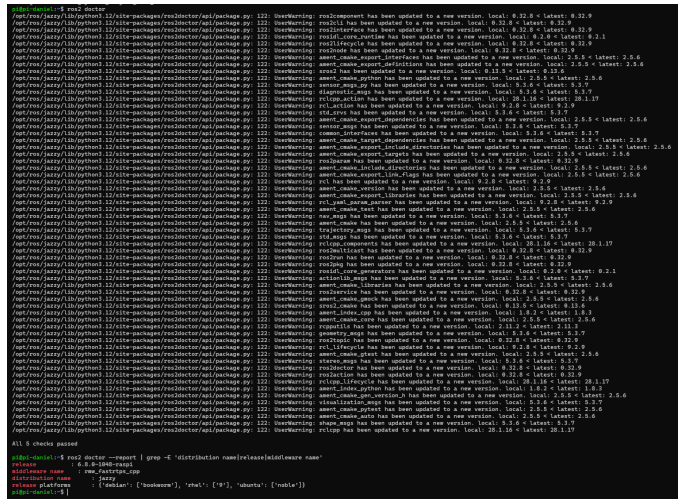

Fig. 2. Hardware platform: `ros2 doctor` reports all five checks passed on the Raspberry Pi 4.

C. Validation Node

A custom ROS 2 Python node, `Validation_Printing_Node_Team_23`, was authored within the `Autonomous_Systems_Project_Team_23` package (ament_python build type). The node creates a 1 Hz timer that logs the message:

> *"ROS 2 Jazzy is running successfully on Team 23 Raspberry Pi"*

Fig. 3 shows 26 seconds of continuous operation on the Raspberry Pi 4.

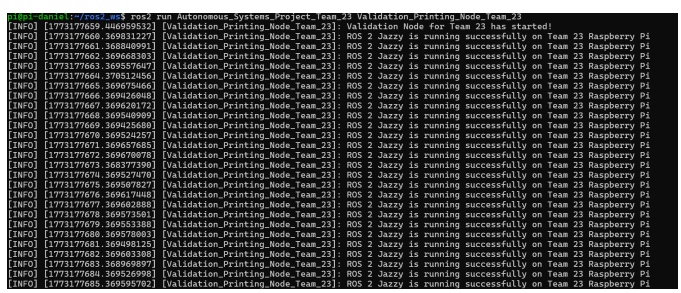

Fig. 3. Hardware validation node executing at 1 Hz: repeated confirmation messages observed over ≈26 s of operation.

V. Simulation Environment Setup and Validation

A. Platform Specification

The simulation platform is an ASUS ROG GA401QE laptop (AMD Ryzen 9 5900HX, NVIDIA GTX 1650). ROS 2 Jazzy and Gazebo Harmonic were installed and bridged via the `ros-jazzy-ros-gz` package.

B. Ackermann Vehicle Model

A SUV-type Ackermann-steered vehicle model was imported into Gazebo Harmonic. The ROS 2–Gazebo bridge exposes the following topics (Table II).

TABLE II
ROS 2–Gazebo Bridge Topics (Simulation)

Topic	Message Type	Direction
`/odom`	Odometry	Gazebo → ROS
`/scan`	LaserScan	Gazebo → ROS
`/camera/image_raw`	Image	Gazebo → ROS
`/tf`	TFMessage	Gazebo → ROS
`/cmd_vel`	Twist	ROS → Gazebo
`/ackermann_cmd`	AckermannDriveStamped	ROS → Gazebo

C. Publisher–Subscriber Node

The `Vehicle_Pub_Sub_Node_Team_23` node publishes a constant velocity command (`linear.x` $= 0.5$ m/s, `angular.z` $= 0.2$ rad/s) to `/cmd_vel` at 1 Hz and subscribes to `/odom` to confirm pose feedback, closing the first software control loop in simulation.

D. Validation Node

A simulation-side validation node, `validation_printing_node`, was executed using the `ros2 daemon` to initialise DDS discovery. The node logged a single confirmation message and exited cleanly. Fig. 4 shows the terminal output on the simulation laptop.

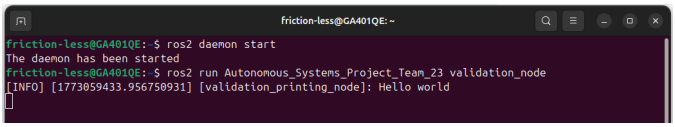

Fig. 4. Simulation environment: ROS 2 daemon started and validation node confirms ROS 2 Jazzy operation on the development laptop.

VI. Results

Table III summarises all Milestone 1 success criteria and their achieved status.

All seven deliverables were achieved. The hardware environment ran `ros2 doctor` with a perfect five-of-five diagnostic score, confirming the ROS 2 Jazzy DDS communication stack is correctly configured. The simulation environment demonstrated bidirectional topic communication between a ROS 2 Python node and a Gazebo Harmonic Ackermann vehicle model, providing the foundation for subsequent milestones.

TABLE III
Milestone 1 Validation Summary

Deliverable	Platform	Status
ROS 2 Jazzy installation	Hardware	✓
ros2 doctor (5/5 checks)	Hardware	✓
Validation node @ 1 Hz	Hardware	✓
ROS 2 Jazzy installation	Simulation	✓
Simulation validation node	Simulation	✓
Ackermann vehicle in Gazebo	Simulation	✓
Publisher–subscriber (/cmd_vel, /odom)	Simulation	✓

VII. Limitations and Future Work

Milestone 1 is limited to environment validation. The following items remain for subsequent milestones:

- **Localization**: integrate wheel-encoder odometry with IMU data via EKF (`robot_localization` package) following [3], [4].
- **Perception**: activate `/scan` (LiDAR) and `/camera/image_raw` topic pipelines for obstacle detection.
- **Navigation**: implement Frenet-frame trajectory planning [5] and an MPC local planner [6].
- **Control**: realise Ackermann-correct steering geometry per (1) in the physical actuator layer.
- **Package cleanup**: align hardware/simulation package metadata (description, maintainer, license) and keep explicit ROS dependency declarations synchronized with imported interfaces.

VIII. Conclusion

This paper presented the complete ROS 2 Jazzy environment validation for Team 23's autonomous Ackermann vehicle project (Milestone 1, MCTR1002 at GUC). The Raspberry Pi 4 hardware platform passed all five `ros2 doctor` diagnostics and ran a 1 Hz validation node for over 26 seconds. The Gazebo Harmonic simulation platform instantiated a full Ackermann SUV model with six ROS 2 topics and executed a publisher–subscriber node commanding and reading pose from the simulated vehicle. These validated environments constitute the communication backbone on which perception, localization, and motion-planning layers will be built in forthcoming milestones.

References

[1] S. Jain *et al.*, "Blockchain and autonomous vehicles: Recent advances and future directions," *IEEE Access*, vol. 9, pp. 130 264–130 328, 2021.

[2] T. Xu *et al.*, "Application of Ackermann steering in obstacle crossing platform of six-wheeled robots," in *Proc. 2nd Int. Symposium on Control Engineering and Robotics (ISCER)*, 2023.

[3] M. Peiris *et al.*, "Kalman filter-based sensor fusion for Ackermann steering mobile robots," *Journal of Physics: Conference Series*, vol. 2811, no. 1, p. 012004, 2024.

[4] I. Georgiev, "Vehicle localization for autonomous driving by sensor fusion," *IEEE Transactions on Intelligent Vehicles*, 2025, dOI not verified from available metadata at staging time.

[5] M. Wang *et al.*, "A real-time dynamic trajectory planning for autonomous driving vehicles," in *Proc. 3rd Conf. on Vehicle Control and Intelligence (CVCI)*, 2019.

[6] H. Wu *et al.*, "MPC-based obstacle avoidance path tracking control for distributed drive electric vehicles," *World Electric Vehicle Journal*, vol. 13, no. 12, 2022.

[7] S. Zhang *et al.*, "Electronic differential control for four-Ackermann-steering-wheeled electric vehicles," *IEEE Transactions on Vehicular Technology*, 2022.